

Guía de Ejercicios Validación de Modelos

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, mandenme un mail a rodriguezr@udesa.edu.ar y lo revisamos.

1. Ejercicios

1.1. Ejercicios Conceptuales

1.1.1. Verdadero o Falso

1. La accuracy es siempre la mejor métrica para evaluar un modelo de clasificación.
2. Un modelo con $AUC = 0.3$ es peor que uno aleatorio.
3. La precisión mide la proporción de verdaderos positivos entre todas las predicciones positivas.
4. El recall y la especificidad suman siempre 1.
5. En un problema médico es más importante maximizar el recall que la precisión.
6. La curva ROC grafica precisión versus recall.
7. Un F1-Score alto garantiza que tanto precisión como recall son altos.
8. La matriz de confusión solo se puede usar para problemas binarios.
9. $AUC = 0.5$ indica un clasificador perfecto.
10. La curva PR es más informativa que ROC cuando las clases están desbalanceadas.
11. Una tasa de falsos positivos alta es siempre peor que una baja.
12. La especificidad es más importante que el recall en sistemas de justicia.

13. El False Discovery Rate y la precisión suman 1.
14. En problemas multiclase no existen falsos positivos.
15. Un modelo puede tener alta accuracy pero baja utilidad práctica.
16. El MSE penaliza más los errores grandes que el MAE.
17. Un R^2 negativo indica que el modelo es peor que predecir la media.
18. El RMSE tiene las mismas unidades que la variable objetivo.
19. El MAPE puede causar problemas cuando hay valores cero en los datos reales.
20. Un modelo con $R^2 = 0.8$ explica el 80 % de la varianza de la variable objetivo.

1.1.2. Verdadero o Falso (Cross Validation)

1. Cross validation permite obtener una estimación más robusta del rendimiento del modelo.
2. En k-fold cross validation, el dataset se divide en k particiones exactamente iguales.
3. Leave-one-out cross validation es siempre la mejor opción para evaluar modelos.
4. Cross validation se puede usar tanto para seleccionar hiperparámetros como para evaluar el rendimiento final.
5. En 5-fold cross validation, cada observación se usa exactamente 4 veces para entrenar y 1 vez para validar.
6. Stratified cross validation es importante para datasets desbalanceados.
7. Cross validation siempre reduce el overfitting del modelo.
8. Un mayor número de folds siempre es mejor en cross validation.

1.1.3. Multiple Choice

1. ¿Cuál es la diferencia principal entre precisión y recall?
 - a) No hay diferencia, son sinónimos
 - b) Precisión se enfoca en predicciones positivas, recall en casos positivos reales
 - c) Recall es para problemas binarios, precisión para multiclase
 - d) Precisión es más importante que recall
2. En un modelo de detección de cáncer, ¿qué métrica priorizarías?
 - a) Accuracy
 - b) Precisión
 - c) Recall
 - d) F1-Score
3. ¿Qué representa un punto en la curva ROC?
 - a) Un modelo diferente
 - b) Un umbral de clasificación diferente
 - c) Una métrica diferente
 - d) Un dataset diferente
4. Si tengo un dataset con 95 % de clase negativa y 5 % de clase positiva, ¿qué métrica puede ser engañosa?
 - a) Accuracy
 - b) Recall
 - c) Precisión
 - d) AUC
5. ¿Cuándo usarías la curva PR en lugar de ROC?
 - a) Siempre
 - b) Nunca

- c)* Cuando las clases están balanceadas
 - d)* Cuando las clases están desbalanceadas
- 6. ¿Qué significa $AUC = 0.8$?
 - a)* 80 % de accuracy
 - b)* 80 % de probabilidad de ranquear correctamente un par positivo-negativo
 - c)* 80 % de precisión
 - d)* 80 % de recall
- 7. ¿Cuál métrica es más sensible a valores atípicos?
 - a)* MSE
 - b)* MAE
 - c)* RMSE
 - d)* R^2
- 8. ¿Qué significa un $R^2 = 0.3$?
 - a)* El modelo explica el 30 % de la varianza
 - b)* El modelo tiene 30 % de accuracy
 - c)* El modelo es 30 % mejor que la media
 - d)* El modelo es 30 % peor que aleatorio
- 9. ¿Cuál es la principal ventaja del MAPE?
 - a)* Es más preciso que otras métricas
 - b)* Es fácil de interpretar como porcentaje
 - c)* No tiene problemas con valores cero
 - d)* Siempre es mejor que el MAE

1.1.4. Multiple Choice (Cross Validation)

1. ¿Cuál es la principal ventaja de cross validation sobre hold-out validation?
 - a) Es más rápido computacionalmente
 - b) Usa toda la data para entrenamiento y validación
 - c) No requiere dividir los datos
 - d) Es más fácil de implementar
2. En 10-fold cross validation, ¿qué porcentaje de los datos se usa para entrenar en cada fold?
 - a) 10 %
 - b) 80 %
 - c) 90 %
 - d) 100 %
3. ¿Cuándo es recomendable usar Leave-One-Out cross validation?
 - a) Con datasets muy grandes
 - b) Con datasets muy pequeños
 - c) Siempre que sea posible
 - d) Nunca
4. ¿Qué es stratified cross validation?
 - a) Una técnica para datasets grandes
 - b) Una forma de mantener la proporción de clases en cada fold
 - c) Un método más rápido de validación
 - d) Una técnica solo para regresión
5. ¿Cuál es el trade-off principal al elegir el número de folds?
 - a) Sesgo vs Varianza
 - b) Velocidad vs Precisión
 - c) Complejidad vs Simplicidad
 - d) Todas las anteriores

1.2. Ejercicios Prácticos

1.2.1. Matriz de Confusión y Métricas Básicas

Un modelo de clasificación binaria para detectar fraude en transacciones tiene los siguientes resultados en un conjunto de prueba de 1000 transacciones:

		Real	
Predicho	Real	45	15
	Predicho	5	935

- Calcular todas las métricas básicas: accuracy, precisión, recall, especificidad, F1-Score.
- ¿Qué puede concluir sobre el rendimiento del modelo?
- ¿Es la accuracy una buena métrica para este problema? Justificar.
- ¿Qué métrica sería más importante para este caso de uso y por qué?

1.2.2. Predicción de Precios

Un modelo para predecir el precio de casas (en miles de dólares) tiene los siguientes resultados en un conjunto de prueba:

Casa	Precio Real	Precio Predicho
1	300	280
2	450	470
3	200	190
4	600	620
5	350	330
6	500	480
7	250	260
8	400	420

- a. Calcular MSE, RMSE, MAE y R^2 para este modelo.
- b. ¿Qué métrica usarías para reportar el rendimiento a un cliente no técnico?
- c. Si una casa cuesta 0 dólares (regalo), ¿qué problema tendrías con el MAPE?
- d. ¿Qué estrategia usarías para manejar este problema?

1.2.3. Comparación de Modelos de Predicción

Tenés dos modelos para predecir ventas mensuales (en miles de unidades):

Modelo A: MSE = 25, MAE = 3.5, $R^2 = 0.85$

Modelo B: MSE = 30, MAE = 3.2, $R^2 = 0.82$

- a. ¿Cuál modelo elegirías si los errores grandes son muy costosos?
- b. ¿Cuál modelo elegirías si todos los errores tienen el mismo costo?
- c. ¿Qué información adicional necesitarías para tomar una decisión más informada?
- d. Si el R^2 del Modelo A fuera 0.95, ¿cambiaría tu respuesta en (a)?

1.2.4. Comparación de Modelos

Tenés dos modelos para clasificar emails como spam o no spam. Los resultados son:

■ **Modelo A:** VP = 180, VN = 820, FP = 30, FN = 20

■ **Modelo B:** VP = 160, VN = 830, FP = 20, FN = 40

- a. Calcular precisión, recall y F1-Score para ambos modelos.
- b. ¿Cuál modelo elegirías y por qué?
- c. ¿Cambiaría tu elección si el costo de un falso negativo fuera 10 veces mayor que el de un falso positivo?

1.2.5. Matriz de Confusión Multiclase

Un modelo clasifica el clima en tres categorías: Soleado (S), Nublado (N), Lluvioso (L). La matriz de confusión es:

		Real		
		S	N	L
Predicho	S	85	10	5
	N	8	82	10
	L	7	12	81

- Calcular la accuracy total del modelo.
- Calcular la precisión, recall y F1-Score para cada clase.
- ¿Qué clase es la más difícil de predecir para el modelo?
- Calcular las métricas macro y micro average.

1.2.6. Curva ROC

Un modelo de clasificación binaria produce las siguientes probabilidades para 8 casos de prueba:

Caso	Clase Real	Probabilidad Positiva
1	Positivo	0.9
2	Negativo	0.8
3	Positivo	0.7
4	Positivo	0.6
5	Negativo	0.4
6	Negativo	0.3
7	Positivo	0.2
8	Negativo	0.1

- Para umbrales 0.5, 0.7 y 0.9, calcular TPR y FPR.
- ¿Qué umbral recomendarías si querés minimizar falsos positivos?
- ¿Qué umbral recomendarías si querés minimizar falsos negativos?
- Estimar el valor aproximado del AUC.

1.2.7. Problema Médico

Un hospital desarrolla un modelo para detectar neumonía en radiografías. El dataset tiene las siguientes características:

- 10,000 radiografías en total
- 500 casos positivos (neumonía)
- 9,500 casos negativos (sin neumonía)

El modelo A tiene una accuracy del 96 % pero detecta solo el 40 % de los casos de neumonía. El modelo B tiene una accuracy del 92 % pero detecta el 85 % de los casos de neumonía.

- Calcular las matrices de confusión aproximadas para ambos modelos.
- ¿Cuál modelo elegirías para uso clínico? Justificar.
- ¿Qué estrategias podrías usar para mejorar el modelo seleccionado?

1.2.8. Implementación en Python

Implementar en Python un análisis completo de validación de modelos que incluya:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.metrics import (classification_report,
8                               confusion_matrix,
9                               roc_curve, auc,
10                              precision_recall_curve)
11
12 # Generar dataset desbalanceado
13 X, y = make_classification(n_samples=1000, n_features=20,
14                           n_classes=2, weights=[0.9, 0.1],
15                           random_state=42)
16
17 # Completar el código para:
18 # 1. Dividir en train/test
19 # 2. Entrenar dos modelos diferentes
20 # 3. Calcular y comparar métricas
21 # 4. Graficar curvas ROC y PR
22 # 5. Crear matrices de confusión
23 # 6. Analizar cual modelo es mejor para este caso
```

1.2.9. Caso de Negocio

Una empresa de e-commerce quiere implementar un sistema de detección de reseñas falsas. Tenés los siguientes datos de rendimiento de tres modelos:

Modelo	Accuracy	Precisión	Recall	AUC
Naive Bayes	0.78	0.65	0.72	0.76
Random Forest	0.82	0.71	0.68	0.81
SVM	0.80	0.73	0.64	0.79

- a. ¿Qué modelo recomendarías si el objetivo principal es no bloquear reseñas legítimas?

- b. ¿Qué modelo recomendarías si el objetivo principal es mantener la calidad alta de reseñas?
- c. ¿Qué información adicional necesitarías para tomar una mejor decisión?
- d. Proponer una estrategia que combine múltiples modelos.

1.3. Ejercicios Prácticos (Cross Validation)

1.3.1. Cálculo de Modelos a Entrenar

Considerar los siguientes escenarios e indicar cuántos árboles es necesario entrenar en cada caso:

1. En el entrenamiento de un modelo CART se usa 5-fold cross validation para seleccionar el hiperparámetro `max_depth` de la siguiente grilla: `[none, 10, 20, 30]`
2. En el entrenamiento de un modelo CART se usa 5-fold cross validation para seleccionar la mejor combinación de los siguientes hiperparámetros:
 - `max_depth`: `[None, 10, 20, 30, 40, 50]`
 - `min_samples_split`: `[2, 5, 10]`
 - `min_samples_leaf`: `[1, 2, 4]`
3. En el entrenamiento de un modelo Random Forest se usa 5-fold cross validation para seleccionar la mejor combinación de los siguientes hiperparámetros:
 - `n_estimators`: `[100, 200, 500, 1000]`
 - `max_depth`: `[10, 20, 30]`
 - `min_samples_split`: `[2, 5, 10]`

1.3.2. Interpretación de Resultados

Tener en cuenta los siguientes resultados de la cross validation del hiperparámetro `n_estimators` para un modelo CART. ¿Qué valor del hiperparámetro elegirían? ¿Por qué?

```

1 {
2 'mean_fit_time': array([7.5903286, 12.5109745, 30.5498374,
3   61.4766215]),
4 'std_fit_time': array([2.8521561, 0.3776906, 0.5143247,
5   0.6323429]),
6 'mean_score_time': array([0.1880916, 0.0871274, 0.9984307,
7   1.9107995]),
8 'std_score_time': array([0.0091651, 0.0154946, 0.1213947,
9   0.1533748]),
10 'param_n_estimators': masked_array(data=[100, 200, 500,
11   1000]),
12 'params': [{'n_estimators': 100},
13   {'n_estimators': 200},
14   {'n_estimators': 500},
15   {'n_estimators': 1000}],
16 'split0_test_score': array([0.7995629, 0.7997190, 0.7998751,
17   0.7998751]),
18 'split1_test_score': array([0.7995629, 0.7997190, 0.7997190,
19   0.7997190]),
20 'split2_test_score': array([0.7995316, 0.7998438, 0.7998438,
21   0.7998438]),
22 'split3_test_score': array([0.7993754, 0.7996877, 0.7996877,
23   0.7998438]),
24 'split4_test_score': array([0.7996877, 0.7998438, 0.7998438,
25   0.7998438]),
26 'mean_test_score': array([0.7995754, 0.799762, 0.7997939,
27   0.7998251]),
28 'std_test_score': array([1.2270604e-04, 6.7250609e-05,
29   7.5455567e-05, 5.4430172e-05]),
30 'rank_test_score': array([4, 3, 2, 1], dtype=int32)
31 }

```

1.3.3. KNN y Cross Validation

Considerar el siguiente conjunto de datos donde se quiere predecir la etiqueta, + o - en función de la variable X. Con este conjunto de datos se quiere entrenar un modelo knn, para el cual el hiperparámetro k se seleccionará por cross-validation:

	1	2	3	4	5	6	7	8	9	10
X	-0.1	0.7	1.0	1.6	2.0	2.5	3.0	3.5	4.1	4.9
Y	-	+	+	-	+	+	-	-	+	+

1. Dar el accuracy de cross-validation por leave-one-out (es decir 10-folds cross-validation) de 1 vecino más cercano para la muestra de entrenamiento.
2. Dar el accuracy de cross-validation por leave-one-out (es decir 10-folds cross-validation) de 3 vecinos más cercanos para la muestra de entrenamiento.
3. ¿Qué valor de k seleccionas?

1.3.4. Implementación de K-Fold

Implementá k-fold cross validation desde cero en Python para evaluar un modelo de regresión lineal:

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn.metrics import mean_squared_error
4
5 def k_fold_cv(X, y, k=5, model_class=LinearRegression):
6     """
7     Implementa k-fold cross validation
8
9     Parameters:
10    X: features
11    y: target
12    k: numero de folds
13    model_class: clase del modelo a evaluar
14
15    Returns:
16    scores: lista de scores para cada fold
17    """
18    # Tu codigo aca
19    pass
20
21 # Datos de ejemplo
22 X = np.random.randn(100, 3)
23 y = X[:, 0] + 2*X[:, 1] - X[:, 2] + np.random.randn(100)*0.1
24
25 # Evaluar modelo
26 scores = k_fold_cv(X, y, k=5)
27 print(f"MSE promedio: {np.mean(scores):.4f}")
28 print(f"Desviacion estandar: {np.std(scores):.4f}")
```

1.3.5. Stratified Cross Validation

Tenemos un dataset desbalanceado con las siguientes clases:

Clase	Cantidad
A	800
B	150
C	50

1. ¿Por qué es importante usar stratified cross validation en este caso?
2. ¿Cuántas observaciones de cada clase habría en cada fold si usamos 5-fold stratified CV?
3. ¿Qué pasaría si usáramos k-fold simple sin estratificación?

1.3.6. Grid Search con Cross Validation

Implementá grid search con cross validation para optimizar hiperparámetros de un SVM:

```
1 from sklearn.svm import SVC
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.datasets import make_classification
4
5 # Generar datos
6 X, y = make_classification(n_samples=1000, n_features=10,
7                           n_classes=2, random_state=42)
8
9 # Definir grilla de parametros
10 param_grid = {
11     'C': [0.1, 1, 10, 100],
12     'gamma': ['scale', 'auto', 0.001, 0.01, 0.1],
13     'kernel': ['rbf', 'linear']
14 }
15
16 # Tu codigo aca para implementar grid search con CV
```

1.3.7. Comparación de Modelos

Usando cross validation, compará el rendimiento de los siguientes modelos en un problema de clasificación:

1. Logistic Regression
2. Random Forest
3. SVM
4. KNN

¿Qué métrica usarías? ¿Cómo determinarías si las diferencias son estadísticamente significativas?

1.3.8. Análisis de Varianza

1. ¿Por qué es importante reportar tanto la media como la desviación estándar de los scores de CV?
2. ¿Qué indica una alta varianza en los scores de cross validation?
3. ¿Cómo podríamos reducir la varianza en los resultados de CV?

2. Soluciones

2.1. Ejercicios Conceptuales

2.1.1. Verdadero o Falso

1. **Falso.** Accuracy puede ser engañosa con clases desbalanceadas.
2. **Verdadero.** $AUC < 0.5$ es peor que aleatorio (0.5).
3. **Verdadero.** $Precisión = VP / (VP + FP)$.
4. **Falso.** Recall + Especificidad no suman 1. Especificidad + FPR suman 1.
5. **Verdadero.** Es más importante no perder casos positivos (enfermos).
6. **Falso.** ROC grafica TPR vs FPR. PR grafica Precisión vs Recall.
7. **Verdadero.** F1 es la media armónica de precisión y recall.
8. **Falso.** También se usa para problemas multiclase.
9. **Falso.** $AUC = 0.5$ indica clasificador aleatorio, no perfecto.
10. **Verdadero.** PR es más sensible a clases minoritarias.
11. **Falso.** Depende del contexto y costos del problema.
12. **Verdadero.** Se prefiere no condenar inocentes.
13. **Verdadero.** $FDR = 1 - Precisión$.
14. **Falso.** Hay múltiples tipos de errores, pero el concepto se extiende.
15. **Verdadero.** Especialmente con clases desbalanceadas.
16. **Verdadero.** MSE eleva al cuadrado los errores, penalizando más los grandes.
17. **Verdadero.** R^2 negativo indica que el modelo es peor que predecir la media.
18. **Verdadero.** $RMSE = \sqrt{MSE}$, mantiene las unidades originales.

19. **Verdadero.** MAPE tiene división por cero cuando $y_i = 0$.
20. **Verdadero.** R^2 mide la proporción de varianza explicada.

2.1.2. Verdadero o Falso (Cross Validation)

1. **Verdadero.** CV proporciona múltiples estimaciones del rendimiento, reduciendo la dependencia de una única división train/test.
2. **Falso.** Los folds pueden tener tamaños ligeramente diferentes si n no es divisible por k .
3. **Falso.** LOOCV puede tener alta varianza y ser computacionalmente costoso.
4. **Verdadero.** CV se usa tanto para model selection como para model evaluation.
5. **Verdadero.** En 5-fold CV, cada observación se usa 4 veces para entrenar y 1 vez para validar.
6. **Verdadero.** Mantiene la proporción de clases en cada fold.
7. **Falso.** CV ayuda a detectar overfitting, pero no lo reduce directamente.
8. **Falso.** Más folds aumenta la varianza y el costo computacional.

2.1.3. Multiple Choice

1. **b)** Precisión se enfoca en predicciones positivas correctas, recall en casos positivos reales detectados.
2. **c)** Recall - es crucial no perder casos de cáncer.
3. **b)** Un umbral de clasificación diferente.
4. **a)** Accuracy - un modelo que predice todo negativo tendría 95 % accuracy.
5. **d)** Cuando las clases están desbalanceadas.
6. **b)** 80 % de probabilidad de ranquear correctamente un par positivo-negativo.

7. **a)** MSE - eleva al cuadrado los errores, amplificando el impacto de valores atípicos.
8. **a)** El modelo explica el 30 % de la varianza de la variable objetivo.
9. **b)** Es fácil de interpretar como porcentaje, aunque tiene problemas con ceros.

2.1.4. Multiple Choice (Cross Validation)

1. **b)** Usa toda la data tanto para entrenamiento como para validación en diferentes momentos.
2. **c)** 90 % (9 de los 10 folds se usan para entrenamiento).
3. **b)** Con datasets muy pequeños, cuando queremos usar todos los datos posibles.
4. **b)** Mantiene la proporción de clases en cada fold.
5. **d)** Todas las anteriores: más folds aumentan sesgo pero reducen varianza, son más lentos pero más precisos, y más complejos pero potencialmente mejores.

2.2. Ejercicios Prácticos

2.2.1. Matriz de Confusión y Métricas Básicas

a) Métricas calculadas:

$$\text{Accuracy} = \frac{45 + 935}{1000} = 0,98$$

$$\text{Precisión} = \frac{45}{45 + 15} = 0,75$$

$$\text{Recall} = \frac{45}{45 + 5} = 0,90$$

$$\text{Especificidad} = \frac{935}{935 + 15} = 0,984$$

$$\text{F1} = 2 \times \frac{0,75 \times 0,90}{0,75 + 0,90} = 0,818$$

- b) El modelo tiene alta accuracy y recall, pero precisión moderada.
- c) No, accuracy puede ser engañosa porque hay pocas transacciones fraudulentas.
- d) Recall es más importante para no perder casos de fraude.

2.2.2. Predicción de Precios

a) Cálculos:

$$\begin{aligned} \text{MSE} &= \frac{1}{8}[(300 - 280)^2 + (450 - 470)^2 + (200 - 190)^2 + (600 - 620)^2 + (350 - 330)^2 \\ &\quad + (500 - 480)^2 + (250 - 260)^2 + (400 - 420)^2] \\ &= \frac{1}{8}[400 + 400 + 100 + 400 + 400 + 400 + 100 + 400] = 325 \end{aligned}$$

$$\text{RMSE} = \sqrt{325} = 18,03$$

$$\text{MAE} = \frac{1}{8}[20 + 20 + 10 + 20 + 20 + 20 + 10 + 20] = 17,5$$

$$R^2 = 1 - \frac{325}{\text{varianza_real}} = 1 - \frac{325}{6500} = 0,95$$

$$\text{Donde: } \bar{y} = \frac{300 + 450 + 200 + 600 + 350 + 500 + 250 + 400}{8} = 381,25$$

$$\text{varianza_real} = (300 - 381,25)^2 + (450 - 381,25)^2 + \dots + (400 - 381,25)^2 = 6500$$

- b) RMSE o MAE - son fáciles de interpretar en las mismas unidades.
- c) División por cero en el denominador del MAPE.
- d) Reemplazar ceros por valor muy pequeño, filtrar ceros, o usar sMAPE.

2.2.3. Comparación de Modelos de Predicción

- a) Modelo A - tiene menor MSE (25 vs 30), penaliza menos los errores grandes.
- b) Modelo B - tiene menor MAE (3.2 vs 3.5), trata todos los errores por igual.
- c) Costos específicos de errores, distribución de los datos, interpretabilidad.
- d) Sí, $R^2 = 0.95$ indicaría que el Modelo A explica mucha más varianza, compensando el mayor MAE.

2.2.4. Comparación de Modelos

Modelo A:

$$\begin{aligned}\text{Precisión} &= \frac{180}{210} = 0,857 \\ \text{Recall} &= \frac{180}{200} = 0,90 \\ \text{F1} &= 0,878\end{aligned}$$

Modelo B:

$$\begin{aligned}\text{Precisión} &= \frac{160}{180} = 0,889 \\ \text{Recall} &= \frac{160}{200} = 0,80 \\ \text{F1} &= 0,842\end{aligned}$$

Modelo A tiene mejor recall y F1. Si los falsos negativos cuestan 10 veces más, definitivamente elegir Modelo A.

2.2.5. Matriz de Confusión Multiclase

a. $\text{Accuracy} = (85 + 82 + 81) / 300 = 0.827$

b. Calculando por clase:

- Soleado: $\text{Precisión} = 0.85$, $\text{Recall} = 0.85$, $\text{F1} = 0.85$
- Nublado: $\text{Precisión} = 0.82$, $\text{Recall} = 0.782$, $\text{F1} = 0.80$
- Lluvioso: $\text{Precisión} = 0.844$, $\text{Recall} = 0.81$, $\text{F1} = 0.827$

c. Nublado es la más difícil (menor F1).

2.2.6. Curva ROC

a. Para cada umbral:

- Umbral 0.5: $\text{TPR} = 0.75$, $\text{FPR} = 0.25$
- Umbral 0.7: $\text{TPR} = 0.75$, $\text{FPR} = 0.25$
- Umbral 0.9: $\text{TPR} = 0.25$, $\text{FPR} = 0.25$

b. Para minimizar FP: umbral 0.9

c. Para minimizar FN: umbral bajo (0.1)

d. AUC aproximado: 0.75

2.2.7. Problema Médico

- a. Modelo A: VP=200, FN=300, FP=100, VN=9400. Modelo B: VP=425, FN=75, FP=725, VN=8775
- b. Modelo B porque detecta más casos de neumonía, crucial en medicina.
- c. Estrategias: datos más balanceados, ensemble, ajuste de umbral.

2.2.8. Implementación en Python

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.datasets import make_classification
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import classification_report,
8   confusion_matrix, roc_curve, auc, precision_recall_curve
9 import matplotlib.pyplot as plt
10
11 # Generar un dataset desbalanceado
12 X, y = make_classification(n_samples=5000, n_features=20,
13   n_informative=2, n_redundant=10,
14   n_clusters_per_class=1, weights
15   =[0.9, 0.1], flip_y=0, random_state=42)
16
17 # Division train/test
18 X_train, X_test, y_train, y_test = train_test_split(X, y,
19   test_size=0.3, stratify=y, random_state=42)
20
21 # Entrenamiento de modelos
22 lr = LogisticRegression()
23 rf = RandomForestClassifier()
24
25 lr.fit(X_train, y_train)
26 rf.fit(X_train, y_train)
27
28 # Predicciones
29 y_pred_lr = lr.predict(X_test)
30 y_pred_rf = rf.predict(X_test)
31 y_proba_lr = lr.predict_proba(X_test)[: ,1]
32 y_proba_rf = rf.predict_proba(X_test)[: ,1]
33
34 # Calculo de metricas
35 print("Logistic Regression")
```

```

32 print(confusion_matrix(y_test, y_pred_lr))
33 print(classification_report(y_test, y_pred_lr, digits=3))
34
35 print("Random Forest")
36 print(confusion_matrix(y_test, y_pred_rf))
37 print(classification_report(y_test, y_pred_rf, digits=3))
38
39 # Curva ROC
40 fpr_lr, tpr_lr, _ = roc_curve(y_test, y_proba_lr)
41 fpr_rf, tpr_rf, _ = roc_curve(y_test, y_proba_rf)
42 auc_lr = auc(fpr_lr, tpr_lr)
43 auc_rf = auc(fpr_rf, tpr_rf)
44
45 plt.figure()
46 plt.plot(fpr_lr, tpr_lr, label="Logistic Reg (AUC = %.2f)" %
47          auc_lr)
48 plt.plot(fpr_rf, tpr_rf, label="Random Forest (AUC = %.2f)" %
49          auc_rf)
50 plt.plot([0,1],[0,1], 'k--')
51 plt.xlabel("FPR")
52 plt.ylabel("TPR")
53 plt.title("Curva ROC")
54 plt.legend()
55 plt.show()
56
57 # Curva Precision-Recall
58 prec_lr, rec_lr, _ = precision_recall_curve(y_test,
59                                             y_proba_lr)
60 prec_rf, rec_rf, _ = precision_recall_curve(y_test,
61                                             y_proba_rf)
62
63 plt.figure()
64 plt.plot(rec_lr, prec_lr, label="Logistic Reg")
65 plt.plot(rec_rf, prec_rf, label="Random Forest")
66 plt.xlabel("Recall")
67 plt.ylabel("Precision")
68 plt.title("Curva Precision-Recall")
69 plt.legend()
70 plt.show()
71
72 # Analisis comparativo
73 # En este caso, mirar F1, recall y precision para la clase
74 # minoritaria (1)
75 # Tambien comparar AUC y la curva PR, que es mas informativa
76 # en datasets desbalanceados

```

2.2.9. Caso de Negocio

- a. Naive Bayes (mayor recall para no bloquear reseñas legítimas).
- b. SVM (mayor precisión para mantener calidad).
- c. Costos de errores, volumen de reseñas, recursos disponibles.
- d. Ensemble que combine todo con pesos según objetivos del negocio.

2.3. Ejercicios Prácticos (Cross Validation)

2.3.1. Cálculo de Modelos a Entrenar

1. **CART con 5-fold CV y 4 valores de max_depth:**
 - 4 valores de hiperparámetro $\times$ 5 folds = **20 árboles**
2. **CART con 5-fold CV y múltiples hiperparámetros:**
 - max_depth: 6 valores
 - min_samples_split: 3 valores
 - min_samples_leaf: 3 valores
 - Total combinaciones: $6 \times 3 \times 3 = 54$
 - 54 combinaciones $\times$ 5 folds = **270 árboles**
3. **Random Forest con 5-fold CV:**
 - n_estimators: 4 valores
 - max_depth: 3 valores
 - min_samples_split: 3 valores
 - Total combinaciones: $4 \times 3 \times 3 = 36$
 - 36 combinaciones $\times$ 5 folds = 180 Random Forests
 - Cada Random Forest tiene su propio n_estimators (100, 200, 500, o 1000)
 - Total de árboles individuales = $180 \times$ promedio de n_estimators
 - Promedio de n_estimators = $(100+200+500+1000)/4 = 450$
 - **Total aproximado: 81,000 árboles individuales**

2.3.2. Interpretación de Resultados

Analizando los resultados:

- **n_estimators = 100:** score = 0.7996, tiempo = 7.59s, std = 1.23e-04
- **n_estimators = 200:** score = 0.7998, tiempo = 12.51s, std = 6.73e-05
- **n_estimators = 500:** score = 0.7998, tiempo = 30.55s, std = 7.55e-05
- **n_estimators = 1000:** score = 0.7999, tiempo = 61.48s, std = 5.44e-05

Recomendación: n_estimators = 100

Justificación:

- La diferencia de score entre 100 y 200 es solo 0.0002 (0.7996 vs 0.7998)
- Esta diferencia es muy pequeña y probablemente no sea estadísticamente significativa
- El tiempo aumenta 65 % (de 7.59s a 12.51s) para una ganancia mínima
- La desviación estándar es ligeramente mayor en 100, pero aún muy baja
- En términos prácticos, 0.0002 de diferencia no justifica el costo computacional adicional

2.3.3. KNN y Cross Validation

Dataset ordenado por X:

Punto	1	2	3	4	5	6	7	8	9	10
X	-0.1	0.7	1.0	1.6	2.0	2.5	3.0	3.5	4.1	4.9
Y	-	+	+	-	+	+	-	-	+	+

a) LOOCV con k=1 (1-NN):

Para cada punto, encontramos su vecino más cercano y vemos si coincide la clase:

- Punto 1 (X=-0.1, Y=-): vecino más cercano es punto 2 (Y=+) → **Error**
- Punto 2 (X=0.7, Y=+): vecino más cercano es punto 3 (Y=+) → **Correcto**
- Punto 3 (X=1.0, Y=+): vecino más cercano es punto 2 (Y=+) → **Correcto**
- Punto 4 (X=1.6, Y=-): vecino más cercano es punto 5 (Y=+) → **Error**
- Punto 5 (X=2.0, Y=+): vecino más cercano es punto 4 (Y=-) → **Error**
- Punto 6 (X=2.5, Y=+): vecino más cercano es punto 5 (Y=+) → **Correcto**
- Punto 7 (X=3.0, Y=-): vecino más cercano es punto 6 (Y=+) → **Error**
- Punto 8 (X=3.5, Y=-): vecino más cercano es punto 7 (Y=-) → **Correcto**
- Punto 9 (X=4.1, Y=+): vecino más cercano es punto 8 (Y=-) → **Error**
- Punto 10 (X=4.9, Y=+): vecino más cercano es punto 9 (Y=+) → **Correcto**

Accuracy k=1: 6/10 = 0.60

b) LOOCV con k=3 (3-NN):

Para cada punto, encontramos sus 3 vecinos más cercanos y usamos votación por mayoría:

- Punto 1: vecinos [2,3,4] → clases [+,-,-] → mayoría - → **Error**
- Punto 2: vecinos [1,3,4] → clases [-,+,-] → mayoría - → **Error**
- Punto 3: vecinos [2,4,5] → clases [+,-,+]
- Punto 4: vecinos [3,5,6] → clases [+,,+]
- Punto 5: vecinos [4,6,7] → clases [-,+,-]
- Punto 6: vecinos [5,7,8] → clases [+,-,-]

- Punto 7: vecinos [6,8,9] → clases [+,-,+] → mayoría + → **Error**
- Punto 8: vecinos [7,9,10] → clases [-,+,+] → mayoría + → **Error**
- Punto 9: vecinos [8,10,7] → clases [-,+, -] → mayoría - → **Error**
- Punto 10: vecinos [9,8,7] → clases [+,-,-] → mayoría - → **Error**

Accuracy k=3: $1/10 = 0.10$

c) **Selección de k:** Elegiría **k=1** porque tiene mejor accuracy (0.60 vs 0.10). Con este dataset pequeño y ruidoso, k=3 incluye demasiados vecinos que añaden ruido a la predicción.

2.3.4. Implementación de K-Fold

```

1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn.metrics import mean_squared_error
4
5 def k_fold_cv(X, y, k=5, model_class=LinearRegression):
6     """
7     Implementa k-fold cross validation
8     """
9     n_samples = len(X)
10    fold_size = n_samples // k
11    indices = np.arange(n_samples)
12    np.random.shuffle(indices) # mezclar indices
13
14    scores = []
15
16    for i in range(k):
17        # Definir indices de validacion
18        start_idx = i * fold_size
19        end_idx = start_idx + fold_size if i < k-1 else
n_samples
20        val_indices = indices[start_idx:end_idx]
21        train_indices = np.concatenate([indices[:start_idx],
22                                       indices[end_idx:]])
23
24        # Dividir datos
25        X_train, X_val = X[train_indices], X[val_indices]
26        y_train, y_val = y[train_indices], y[val_indices]
27

```

```

28     # Entrenar modelo
29     model = model_class()
30     model.fit(X_train, y_train)
31
32     # Predecir y evaluar
33     y_pred = model.predict(X_val)
34     score = mean_squared_error(y_val, y_pred)
35     scores.append(score)
36
37     return scores

```

2.3.5. Stratified Cross Validation

1. **Importancia de stratified CV:** Con clases desbalanceadas, k-fold simple podría crear folds sin representación de clases minoritarias. Stratified CV garantiza que cada fold mantenga la proporción original de clases.
2. **Observaciones por fold en 5-fold stratified CV:**
 - Total: 1000 observaciones
 - Proporción: A=80 %, B=15 %, C=5 %
 - Por fold (200 obs): A=160, B=30, C=10
3. **Problema sin estratificación:** Algunos folds podrían tener muy pocas o ninguna observación de clases B y C, dando estimaciones sesgadas del rendimiento.

2.3.6. Grid Search con Cross Validation

```

1 from sklearn.svm import SVC
2 from sklearn.model_selection import GridSearchCV
3 from sklearn.datasets import make_classification
4
5 # Generar datos
6 X, y = make_classification(n_samples=1000, n_features=10,
7                           n_classes=2, random_state=42)
8
9 # Definir grilla de parametros
10 param_grid = {
11     'C': [0.1, 1, 10, 100],

```

```

12     'gamma': ['scale', 'auto', 0.001, 0.01, 0.1],
13     'kernel': ['rbf', 'linear']
14 }
15
16 # Grid search con CV
17 grid_search = GridSearchCV(
18     SVC(),
19     param_grid,
20     cv=5,
21     scoring='accuracy',
22     n_jobs=-1,
23     verbose=1
24 )
25
26 grid_search.fit(X, y)
27
28 print(f"Mejores parametros: {grid_search.best_params_}")
29 print(f"Mejor score: {grid_search.best_score_:.4f}")

```

2.3.7. Comparación de Modelos

```

1 from sklearn.model_selection import cross_val_score
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.svm import SVC
5 from sklearn.neighbors import KNeighborsClassifier
6 import numpy as np
7
8 # Modelos a comparar
9 models = {
10     'Logistic Regression': LogisticRegression(),
11     'Random Forest': RandomForestClassifier(),
12     'SVM': SVC(),
13     'KNN': KNeighborsClassifier()
14 }
15
16 # Comparar modelos
17 results = {}
18 for name, model in models.items():
19     scores = cross_val_score(model, X, y, cv=5, scoring='
f1_weighted')
20     results[name] = scores
21     print(f"{name}: {scores.mean():.4f} (+/- {scores.std()
*2:.4f})")

```

```

22
23 # Test estadístico (t-test pareado) para comparar modelos
24 from scipy import stats
25 model1_scores = results['Random Forest']
26 model2_scores = results['SVM']
27 t_stat, p_value = stats.ttest_rel(model1_scores,
28                                   model2_scores)
29 print(f"p-value: {p_value:.4f}")

```

Usaría **F1-score weighted** para datasets desbalanceados o **accuracy** para balanceados. Para significancia estadística, usaría t-test pareado entre scores de CV.

2.3.8. Análisis de Varianza

1. **Importancia de media y desviación:** La media indica el rendimiento esperado, la desviación indica la estabilidad del modelo. Ambas son cruciales para tomar decisiones informadas.

2. **Alta varianza indica:**

- Modelo sensible a cambios en datos de entrenamiento
- Posible overfitting
- Dataset pequeño o muy heterogéneo
- Necesidad de regularización

3. **Reducir varianza:**

- Aumentar el dataset
- Usar modelos más simples
- Aplicar regularización
- Aumentar el número de folds (trade-off con costo computacional)
- Usar stratified CV para datasets desbalanceados